

Machine Learning & Deep Learning

A Brief Review

Foundations for Full-Stack AI Applications

TCHAPGA TCHITO C. | University of Buea — COT | 2025/2026

Supervised

Unsupervised

Regression

Classification

Neural Nets

CNN

RNN

Transformers

2

Main Branches

10+

Core Algorithms

**DL $\subset$
ML**

Relationship

What We Cover — Two Sections



Machine Learning

- What is ML?
- Supervised vs Unsupervised vs RL
- Key Algorithms (RF, SVM, KNN...)
- Model Evaluation & Cross-Validation
- Scikit-learn Pipeline Workflow

Deep Learning

- What is Deep Learning?
- Perceptron & MLP
- Convolutional Neural Networks
- RNNs & LSTMs
- Keras / TensorFlow API

SECTION

Machine Learning

Supervised · Unsupervised · Reinforcement · Evaluation · Scikit-learn

M



What is Machine Learning?

"A system that learns from data to improve performance on a task without being explicitly programmed."

— Tom Mitchell, 1997

S

Supervised

Learns from labeled data to predict outputs.
Ex: spam filter, price prediction, IRIS classifier

U

Unsupervised

Finds patterns/structure in unlabeled data.
Ex: customer clustering, PCA dimensionality reduction

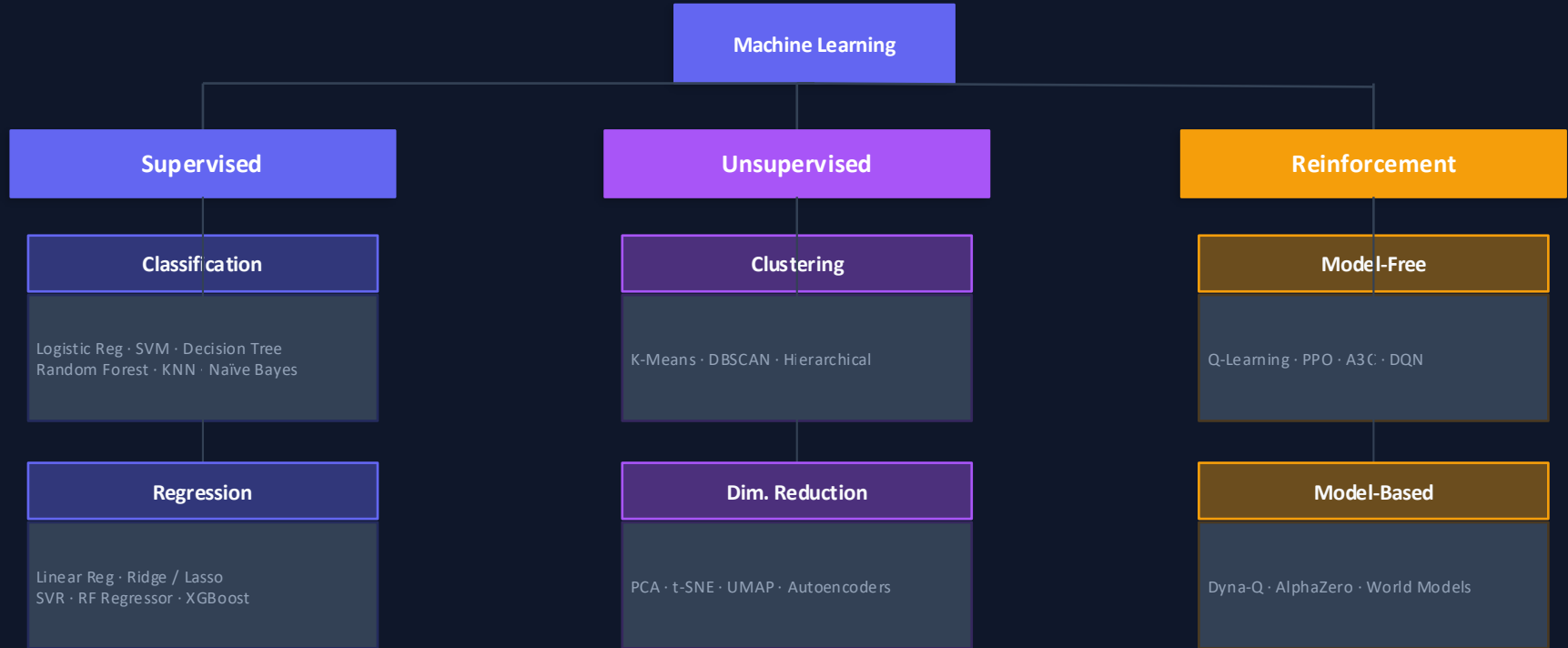
R

Reinforcement

Agent learns via reward signals from environment.
Ex: game-playing AI, robot navigation, AlphaGo

ML market: \$79B (2022) → \$500B+ (2030) | 90% of AI research now involves ML

ML Algorithm Taxonomy



Key ML Algorithms — At a Glance

LR

Linear Regression

Regression

Fits a hyperplane to minimize MSE.
Predicts continuous values.

LG

Logistic Regression

Classification

Sigmoid maps scores to probability.
Binary or multinomial output.

DT

Decision Tree

Both

Recursive if-else splits on features.
Highly interpretable, prone to overfit.

RF

Random Forest

Best Tabular

Ensemble of N trees with bagging.
Robust, handles missing values, feature importance.

SVM

Support Vector Machine

Classification

Maximises margin between classes.
Kernel trick for non-linear boundaries.

KM

K-Means Clustering

Unsupervised

Assigns points to K centroids iteratively.
Requires choosing K; sensitive to init.

Random Forest — Deep Dive (IRIS Use-Case)

1

Bootstrap Sampling — draw N random subsets (with replacement) from training data

2

Grow N Decision Trees — each tree trained on a different bootstrap sample

3

Random Feature Selection — at each split, consider only $\sqrt{p}$ features

4

Majority Vote (classification) or Average (regression) across all N trees

5

Feature Importance — mean decrease in Gini impurity across all trees & splits

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

iris = load_iris()
X, y = iris.data, iris.target
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

clf = RandomForestClassifier(
    n_estimators=100, max_depth=None,
    random_state=42, n_jobs=-1)
clf.fit(X_tr, y_tr)

preds = clf.predict(X_te)
print(f"Accuracy: {accuracy_score(y_te, preds):.2%}")
print(classification_report(y_te, preds,
    target_names=iris.target_names))

# Feature importance
fi = pd.Series(clf.feature_importances_,
    index=iris.feature_names).sort_values()
fi.plot(kind='barh', color='#F59E0B')
```

Typical IRIS accuracy with 100 trees: 96–100%

n_estimators

max_depth

min_samples

max_features

bootstrap

Model Evaluation Metrics

Classification Metrics

Accuracy

$$(TP+TN) / \text{Total}$$

Overall correctness

Precision

$$TP / (TP+FP)$$

Of predicted +, how many correct?

Recall

$$TP / (TP+FN)$$

Of actual +, how many found?

F1-Score

$$2 \cdot P \cdot R / (P+R)$$

Harmonic mean — use for imbalanced data

ROC-AUC

Area under ROC curve

Threshold-independent performance

TP	FP
FN	TN

Confusion Matrix

Regression Metrics

- MAE = $\text{mean} |y - \hat{y}|$ — robust to outliers
- MSE = $\text{mean}(y - \hat{y})^2$ — penalises large errors
- RMSE = $\sqrt{\text{MSE}}$ — same units as target
- $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ — 1.0 = perfect

Bias-Variance Tradeoff

Underfitting

High Bias, Low Var — model too simple, fails on train data

Good Fit

Balanced — generalises well to unseen data

Overfitting

Low Bias, High Var — memorises train, fails on test

Cross-Validation & Regularization

K-Fold Cross-Validation (k=5)

VAL	TRAIN	TRAIN	TRAIN	TRAIN	Fold 1
TRAIN	VAL	TRAIN	TRAIN	TRAIN	Fold 2
TRAIN	TRAIN	VAL	TRAIN	TRAIN	Fold 3
TRAIN	TRAIN	TRAIN	VAL	TRAIN	Fold 4
TRAIN	TRAIN	TRAIN	TRAIN	VAL	Fold 5

Score = mean of 5 validation scores → more reliable than single train/test split

Regularization

Ridge (L2)

$$L + \lambda \sum w_i^2$$

Shrinks all weights; rarely zero. Best when all features are relevant.

Lasso (L1)

$$L + \lambda \sum |w_i|$$

Drives irrelevant weights to zero → automatic feature selection.

Elastic Net

$$L + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$

Combines both; handles correlated features gracefully.

Hyperparameter Tuning

GridSearchCV

exhaustive

RandomizedSearchCV

fast sampling

Optuna

Bayesian TPE

Scikit-learn Pipeline Workflow

Raw Data

Preprocess

Feature Eng.

Model

Evaluate

Deploy

```
pandas / numpy
StandardScaler
LabelEncoder
PCA
SelectKBest
RF / SVM
LogisticReg
cross_val_score
ROC-AUC
joblib.dump
Flask/Django

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.datasets import load_iris
import joblib

X, y = load_iris(return_X_y=True)

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', RandomForestClassifier(n_estimators=100, random_state=42))
])

# Cross-validate
scores = cross_val_score(pipe, X, y, cv=5, scoring='accuracy')
print(f"CV Accuracy: {scores.mean():.3f} ± {scores.std():.3f}")

# Hyperparameter tuning
param_grid = {'clf__n_estimators': [50, 100, 200], 'clf__max_depth': [None, 5, 10]}
gs = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1)
gs.fit(X, y)
print("Best params:", gs.best_params_)

# Save for deployment
joblib.dump(gs.best_estimator_, "iris_rf_pipeline.joblib")
```

SECTION

Deep Learning

Neural Networks · CNNs · RNNs · Keras / TensorFlow



What is Deep Learning?

Artificial Intelligence

Machine Learning

Deep Learning

Multi-layer neural networks
learning hierarchical features
directly from raw data

Key Capabilities:



Images

CNNs — Computer Vision



Text/Speech

RNNs / Transformers — NLP, ASR



Tabular

MLP — classification, regression



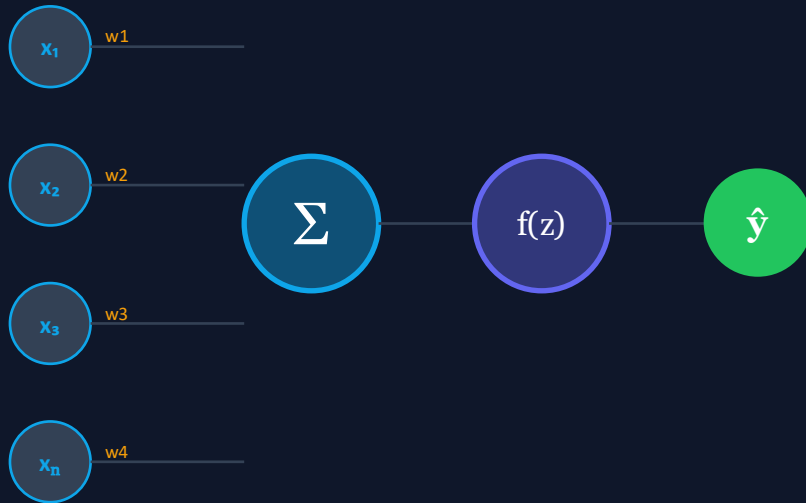
Generation

GANs / VAEs / Diffusion Models

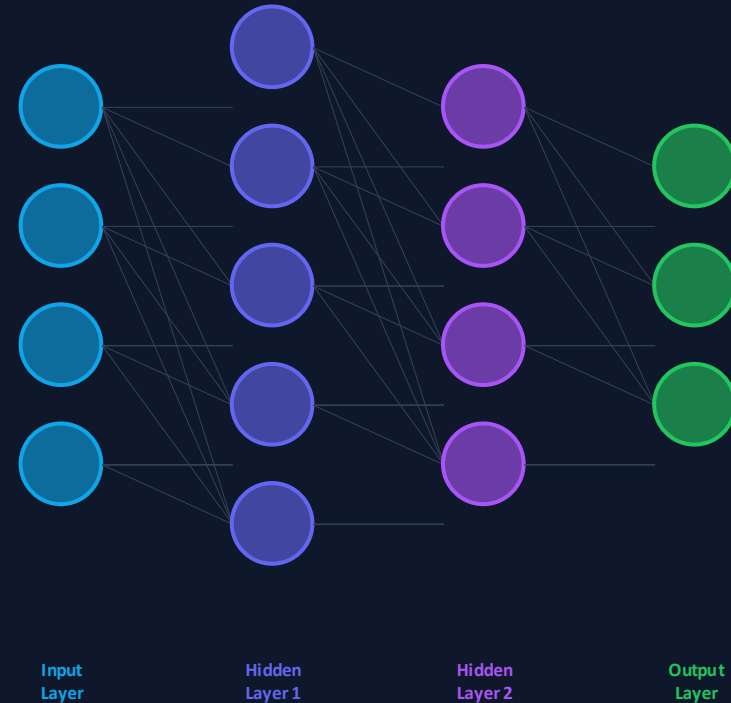
Timeline:



The Neuron & Multilayer Perceptron (MLP)



MLP Architecture



Sigmoid	Tanh	ReLU	Softmax
$1/(1+e^{-z})$	$(e^z - e^{-z}) / (e^z + e^{-z})$	$\max(0, z)$	$e^z_i / \sum e^z_j$
Output binary	Hidden, centred	Hidden, fast	Multiclass out

Backpropagation & Gradient Descent

Cross-Entropy Loss

$$L = -(1/N) \sum_i y_i \cdot \log(\hat{y}_i)$$

Gradient Descent

$$w \leftarrow w - \eta \cdot \partial L / \partial w$$

Chain Rule

$$\partial L / \partial w = \partial L / \partial a \cdot \partial a / \partial z \cdot \partial z / \partial w$$

Batch Norm

$$\hat{x} = (x - \mu) / \sigma \quad \text{then} \quad \gamma \hat{x} + \beta$$

Optimizer Comparison



SGD

Simple, noisy, requires careful LR tuning.
Often needs momentum (0.9).



Adam

Adaptive LR per parameter. Fast convergence.
Defaults: lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$.



AdamW

Adam + weight decay (decoupled L2).
Best default for most deep learning tasks.



```
model.compile(optimizer=tf.keras.optimizers.AdamW(lr=1e-3),  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
model.fit(X_tr, y_tr, epochs=50, validation_split=0.2)
```

Convolutional Neural Networks (CNNs)



Convolutional Layer

Learnable filters (kernels) slide over the input and compute dot products → feature maps that detect edges, textures, shapes.

Pooling Layer

Reduces spatial dimensions (width × height) while retaining dominant features. Provides translation invariance.

Fully Connected

Flattened features classified by dense layers. Final softmax gives class probabilities.

Landmark Architectures

LeNet-5	1998	1.0M params
AlexNet	2012	60M params
VGG-16	2014	138M params
ResNet-50	2015	25M params
EfficientNetB0	2019	5.3M params

Used in CIFAR-10 Django use-case of this course (32×32×3 input → 3-block CNN → 10 class output)

CIFAR-10 CNN — Deep Dive (Django Use-Case)



```
from tensorflow.keras import Sequential, layers

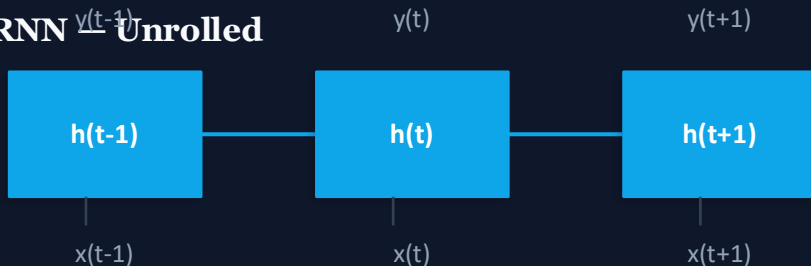
model = Sequential([
    # Block 1
    layers.Conv2D(32,(3,3),activation='relu',
                  padding='same',input_shape=(32,32,3)),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.25),
    # Block 2
    layers.Conv2D(64,(3,3),activation='relu',padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.25),
    # Classifier head
    layers.Flatten(),
    layers.Dense(256,activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10,activation='softmax'), # 10 classes
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.summary() # ~700K trainable params
```

CIFAR-10 Classes

0 airplane	1 automobile
2 bird	3 cat
4 deer	5 dog
6 frog	7 horse
8 ship	9 truck
60k images	32×3 input shape 2×3
10 classes	50k/10k train / test

RNNs & LSTMs — Sequence Modelling

RNN $y(t-1)$ **Unrolled**



LSTM — Long Short-Term Memory

- Forget gate — decides what to discard from cell state
- Input gate — decides what new info to store
- Output gate — decides what to output from cell state

Use cases: Time-series · Text generation · Speech recognition · Music generation



```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM, Dense

T, features = 30, 10 # sequence length, features

model = Sequential([
    LSTM(64, return_sequences=True,
        input_shape=(T, features)),
    LSTM(32),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# note: superseded by Transformers for NLP
# but still state-of-art for short time series
```

Largely superseded by Transformers for NLP tasks (2017+),
but still strong for univariate/multivariate time-series.

Transfer Learning & Fine-Tuning

1

Pre-train on large dataset

Model trained on ImageNet (1.2M images, 1000 classes) — learns universal visual features.

2

Freeze base layers

Load pre-trained weights; set `base.trainable=False`. Only classifier head is updated.

3

Train classifier head

Add `GlobalAveragePooling` + `Dense` layers. Train with normal LR for new task.

4

Fine-tune (optional)

Unfreeze top N layers. Use a very small LR ($1e-5$) to adjust high-level features.

```
import tensorflow as tf

base = tf.keras.applications.MobileNetV2(
    input_shape=(224,224,3),
    include_top=False,
    weights='imagenet')
base.trainable = False # freeze

x = base.output
x = tf.keras.layers.GlobalAveragePooling2D()(x)
out = tf.keras.layers.Dense(
    num_classes, activation='softmax')(x)
model = tf.keras.Model(base.input, out)

# Fine-tune: unfreeze last 30 layers
base.trainable = True
for layer in base.layers[:-30]:
    layer.trainable = False
model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss='categorical_crossentropy')
```

Popular Pre-Trained Models

MobileNetV2	3.4M	71.8%	mobile/edge
ResNet-50	25.5M	74.9%	general
EfficientNetB0	5.3M	77.1%	efficient
VGG-16	138M	72.4%	classic

Keras / TensorFlow API Cheatsheet



Model Building

- `Sequential(layers)` – stack of layers in order
- `Model(inputs, outputs)` – Functional API, multiple paths
- `model.summary()` – print architecture & param count
- `Dense(units, activation)` – fully connected
- `Conv2D(filters, kernel, padding, strides)`
- `LSTM(units, return_sequences)` – recurrent
- `Dropout(rate)` – regularization
- `BatchNormalization()` – stabilise training
- `Flatten()` / `GlobalAveragePooling2D()`

Training & Evaluation

- `model.compile(optimizer, loss, metrics)`
- `model.fit(X, y, epochs, batch_size, validation_split)`
- `model.evaluate(X_test, y_test)`
- `model.predict(X_new)` – raw probabilities
- `EarlyStopping(patience=5, restore_best_weights=True)`
- `ReduceLROnPlateau(factor=0.5, patience=3)`
- `ModelCheckpoint('best.keras', save_best_only=True)`
- `model.save('model.keras')` / `load_model('model.keras')`

Machine Learning vs Deep Learning



Criterion	Machine Learning	Deep Learning
Data requirements	Works well with small/medium datasets	Needs large datasets (10k–1M+)
Feature engineering	Manual selection required	Learns features automatically
Training speed	Fast — seconds to minutes	Slow — hours to days (needs GPU)
Hardware	CPU is sufficient	GPU / TPU highly recommended
Interpretability	High — RF importance, SHAP values	Low — black box, needs XAI
Best for	Tabular / structured data	Images, text, audio, video
Python library	scikit-learn	Keras / TensorFlow / PyTorch
Course use-case	IRIS + Random Forest (Flask API)	CIFAR-10 + CNN (Django API)

Full-Stack AI Pipeline — End-to-End



Track A — Flask + Random Forest + IRIS

- Tabular CSV (4 features)
- StandardScaler + RF
- joblib pipeline
- /predict POST route
- HTML form → JSON response

Track B — Django + CNN + CIFAR-10

- 32×32 RGB image upload
- Data augmentation + 3-block CNN
- model.save('.keras')
- DRF APIView → PIL decode → predict
- JSON class label + confidence

The full-stack practicals in this course integrate both tracks end-to-end.

Practical Tips & Common Pitfalls

Do ✓

- ✓ Always normalise / standardise features before training
- ✓ Use train / validation / test split — never two-way split
- ✓ Monitor validation loss — stop when it diverges from train loss
- ✓ Use callbacks: EarlyStopping + ReduceLROnPlateau
- ✓ Save model artifacts with versioning (v1, v2...)
- ✓ Set random seeds for reproducibility (`np.random.seed`, `tf.random.set_seed`)

Don't ✗

- ✗ Don't leak test data into training (data leakage = inflated metrics)
- ✗ Don't ignore class imbalance — use `class_weight` or SMOTE
- ✗ Don't skip cross-validation for small datasets
- ✗ Don't use sigmoid in hidden layers (vanishing gradients) — use ReLU
- ✗ Don't tune on test set — use a held-out validation set
- ✗ Don't deploy without monitoring prediction distributions in production

Quick Math Reference



Mean Squared Error

$$L = (1/N) \sum_i (y_i - \hat{y}_i)^2$$

Binary Cross-Entropy

$$L = -(1/N) \sum_i [y_i \log \hat{y}_i + (1-y_i) \log(1-\hat{y}_i)]$$

Gradient Descent

$$w \leftarrow w - \eta \cdot \nabla_w L(w)$$

Backprop weight update

$$\Delta w = -\eta \cdot \partial L / \partial w \quad (\text{chain rule})$$

Convolution

$$(f \star g)(t) = \sum_{\tau} f(\tau) \cdot g(t - \tau)$$

Softmax

$$P(y=k \mid x) = \exp(z_k) / \sum_j \exp(z_j)$$

Adam first moment

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

Precision & Recall

$$P = TP / (TP + FP) \quad R = TP / (TP + FN) \quad F1 = 2PR / (P + R)$$

Recommended Learning Resources



Books

 Hands-On ML (Géron, 3rd ed.)

 Deep Learning (Goodfellow et al.)

 Python ML (Raschka & Mirjalili)

 Dive into Deep Learning (d2l.ai)

Courses

 fast.ai — Practical Deep Learning

 DeepLearning.AI (Coursera)

 CS231n — Stanford CNN

 Full Stack Deep Learning (FSDL)

Libraries

 scikit-learn.org — ML docs

 keras.io — Keras guides

 pytorch.org — PyTorch tutorials

 huggingface.co — models & datasets

Datasets

 kaggle.com/datasets

 archive.ics.uci.edu

 tensorflow.org/datasets

 roboflow.com/universe

This course uses scikit-learn + TensorFlow / Keras for all ML & DL practicals

Key Takeaways

ML vs DL

ML learns from structured data; DL learns hierarchical features from raw data via deep networks.

Random Forest

Best default for tabular data — ensemble of trees with bagging. IRIS use-case in this course.

CNNs

Spatial feature learning via filters. 3-block Conv+Pool+BN+Dropout architecture for CIFAR-10.

Train/Val/Test

Always use 3-way split. Use cross-validation for small datasets. Never tune on the test set.

Evaluation

Accuracy alone is insufficient — report F1, confusion matrix, ROC-AUC, especially for imbalanced data.

Transfer Learning

Fine-tune pre-trained models (MobileNetV2, ResNet-50) to save data, compute, and training time.

Deployment

Save RF models with joblib; Keras models as .keras files. Serve via Flask routes or Django DRF views.

Regularization

Use Dropout, BatchNorm, EarlyStopping, ReduceLROnPlateau, and L2/Adam weight decay to prevent overfit.